STEats — Équipe J

Rendu D1 Backend Java / Maven

Groupe 13 — Conception orientée objet — SI4 — 2026–2027

Analyse de l’archive develop fournie le 11 octobre 2026. Identifiant indiqué dans le commentaire ZIP : 8dfc5190fccc50e5b58f17bb0700c56eddc0fd09. Cet identifiant ne remplace pas la vérification du tag Git BACKEND sur main.

Membre (nom et prénom — D0)	Rôle principal	Exigence stratégique
ADJEGAN Justice	Quality Assurance (QA)	A3* — Supervision
ATI-ATCHA Wahid	Software Architect	R6* — Offres / fidélité
LOUBARIS Adam	External Integration Manager	R5* — Capacités
MOROU Essognina Fadilou	Product Owner (PO)	D5* — Incidents
TSOULI Younes	Ops	AI3* — Recommandations

Version technique documentée, à compléter avant dépôt. Les identités et exigences référentes sont reprises du D0 fourni (PS7-Sheet (2).pdf). Les rôles et les 100 points par membre sont confirmés par l’équipe. Les classements de maîtrise, le bilan de collaboration et les preuves SonarQube restent à renseigner. Aucun résultat personnel ou indicateur SonarQube n’est inventé.

Table des matières

1. Périmètre fonctionnel … p. 3

2. Périmètre de compétence … p. 4

3. Conception UML du cœur fonctionnel … p. 5

4. Qualité du code, tests et gestion de projet … p. 9

5. D5* — Incidents de livraison … p. 10

6. R5* — Capacités et créneaux … p. 11

7. R6* — Offres et promotions … p. 12

8. A3* — Supervision des livraisons … p. 13

9. AI3* — Recommandations … p. 14

10. Auto-évaluation … p. 15

Périmètre des preuves

L’archive develop contient le code et les configurations, sans historique .git ni résultat SonarQube. Le D0 Team-J a été fourni séparément sous le nom PS7-Sheet (2).pdf : les exigences stratégiques et D4 ci-dessous sont comparées à ces engagements. Le présent D1 décrit l’implémentation observée, et non les seules intentions de conception.

DONE signifie que le sous-périmètre identifié est implémenté et validé par les tests exécutés. WIP signale une couverture fonctionnelle partielle ou des limites importantes ; BLOCKED signale une fonctionnalité absente. L’isolation est évaluée par rapport aux défaillances des modules périphériques, et non par rapport à une panne du stockage propre au composant.

1. Périmètre fonctionnel

Exigence	État	Points forts	Points faibles	Peut être isolé fonctionnellement
C2 — Filtrage	DONE	Prix inclusifs, cuisines, type ; RestaurantFilterTest et RestaurantServiceTest.	Constructeur court de RestaurantService inutilisable pour findRestaurants (dépendances nulles).	Oui
C4 — Commandes précédentes	WIP	findByUserId ; classement par nombre de commandes non annulées.	D0 : consulter l'historique ; repository présent, mais pas de façade getOrderHistory ni de test de consultation dédié.	Oui
C5 — Panier / commande	WIP	Fusion des quantités ; stockage du panier ; lieu et horaire de livraison.	Pas de contrôle du changement de restaurant d'un panier existant ; stock non décrémenté.	Oui
C6 — Validation	WIP	Refus du panier vide, lieu/horaire inconnus, plat indisponible sur le parcours avec livraison.	createOrder sans livraison ne contrôle pas disponibilité/stock ; pas de validation utilisateur dans OrderService.	Non
D1 — Connexion livraison	WIP	Gateway injectable ; requête validée ; exception convertie en échec.	Adaptateur simulé ; earliest ready time R5* et créneaux retournés par le fournisseur absents.	Oui
D3 — Référence livraison	DONE	Référence D-UUID ; DeliveryInfo sauvegardé après succès.	Pas de garde d'idempotence ni de validation d'une référence retournée vide.	Oui
D4 — Suivi livraison	WIP	ACCEPTED / IN_TRANSIT / DELIVERED ; commande synchronisée.	D0 : 8 statuts et historique horodaté ; seuls 4 statuts présents, sans historique.	Oui
D5* — Incidents	WIP	FAILED et motif ; commande conservée PAID/READY.	Aucun scénario actif d'incident ; pas de remboursement / relivraison.	Oui
P1 — Paiement	DONE	Paiement simulé et crédit ; refus, exception, réponse nulle testés.	Pas de banque réelle ; paiement répété non interdit.	Oui
P2 — Enregistrement	DONE	Commande PENDING avec lignes, total, lieu et horaire.	Stockage volatil ; absence de transaction avec consommation de capacité.	Oui
R3 — Gestion plats	DONE	Ajout / modification nom et description ; doublon et champs vides refusés.	Pas de contrôle d'autorisation restaurateur ; feature R3 vide.	Oui
R5* — Capacités / temps	WIP	Capacité par TimeSlot, décrément, refus si pleine.	Pas d'atomicité, restitution après échec, estimation du temps ou réservation avancée.	Non
R6* — Offres	BLOCKED	Aucune implémentation identifiée.	Pas de modèle promotion ni règle de remise.	Non
A3* — Supervision	BLOCKED	Statut unitaire disponible dans DeliveryInfo.	Pas de tableau de supervision, agrégation ni service administrateur.	Non
AI3* — Recommandations	WIP	Premier ajout, filtrage, déduplication, repli si panne.	Diversité minimale D0 non contrôlée ; IA simulée ; code de recommandation dupliqué.	Oui

2. Périmètre de compétence

Les cinq membres doivent classer les 15 exigences : chaque colonne doit être une permutation de 1 à 15, de la moins maîtrisée à la mieux maîtrisée. Le classement individuel ne mesure ni le nombre de commits ni le rôle déclaré ; il doit être renseigné personnellement.

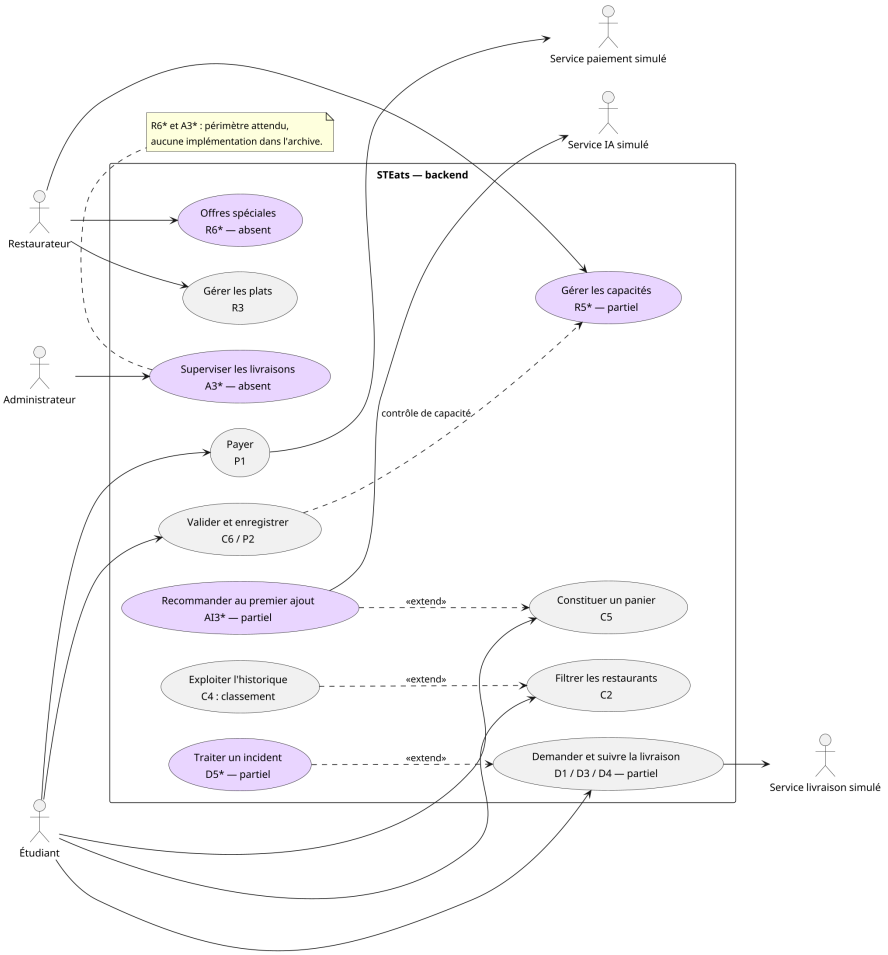
Les colonnes suivent l'ordre alphabétique des noms du D0 : ADJEGAN, ATI-ATCHA, LOUBARIS, MOROU, TSOULI. Aucune auto-évaluation n'a été déduite du code ; chaque membre doit renseigner sa propre colonne.

Exigence	État	ADJEGAN Justice	ATI-ATCHA Wahid	LOUBARIS Adam	MOROU E. Fadilou	TSOULI Younes
C2	DONE	À saisir	À saisir	À saisir	À saisir	À saisir
C4	WIP	À saisir	À saisir	À saisir	À saisir	À saisir
C5	WIP	À saisir	À saisir	À saisir	À saisir	À saisir
C6	WIP	À saisir	À saisir	À saisir	À saisir	À saisir
D1	WIP	À saisir	À saisir	À saisir	À saisir	À saisir
D3	DONE	À saisir	À saisir	À saisir	À saisir	À saisir
D4	WIP	À saisir	À saisir	À saisir	À saisir	À saisir
D5*	WIP	À saisir	À saisir	À saisir	À saisir	À saisir
P1	DONE	À saisir	À saisir	À saisir	À saisir	À saisir
P2	DONE	À saisir	À saisir	À saisir	À saisir	À saisir
R3	DONE	À saisir	À saisir	À saisir	À saisir	À saisir
R5*	WIP	À saisir	À saisir	À saisir	À saisir	À saisir
R6*	BLOCKED	À saisir	À saisir	À saisir	À saisir	À saisir
A3*	BLOCKED	À saisir	À saisir	À saisir	À saisir	À saisir
AI3*	WIP	À saisir	À saisir	À saisir	À saisir	À saisir

À vérifier avant dépôt : chaque colonne utilise une fois chacun des rangs 1 à 15 ; les réponses reflètent la maîtrise réelle du membre, indépendamment de l'exigence dont il est référent.

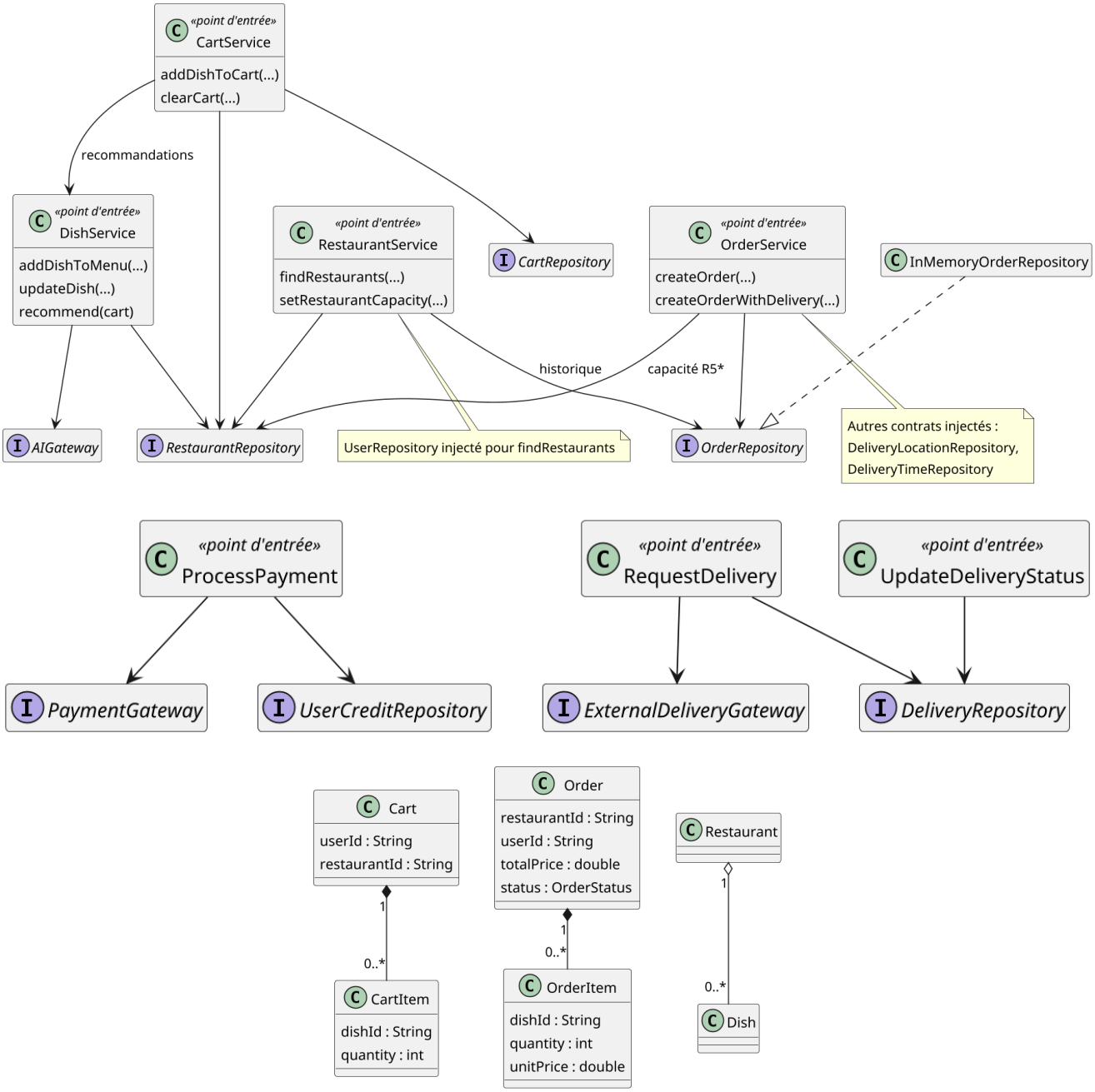
3. Conception UML du cœur fonctionnel

3.1 Diagramme de cas d’utilisation global



Violet : exigences stratégiques. R6* et A3* sont montrées comme fonctions attendues absentes. C4 est limité au classement par historique ; AI3* est une extension déclenchée au premier ajout au panier.

3.2 Diagrammes de classes du cœur



Les interfaces des gateways constituent les points de branchement périphériques. R5* n'a pas de contrat autonome : OrderService lit et modifie directement la capacité de Restaurant. Ce couplage explique l'absence d'isolation fonctionnelle de R5*. Les noms courts représentent les classes réelles ; les implémentations répétitives des repositories sont omises pour la lisibilité.

3.2 Diagrammes de classes — choix de conception

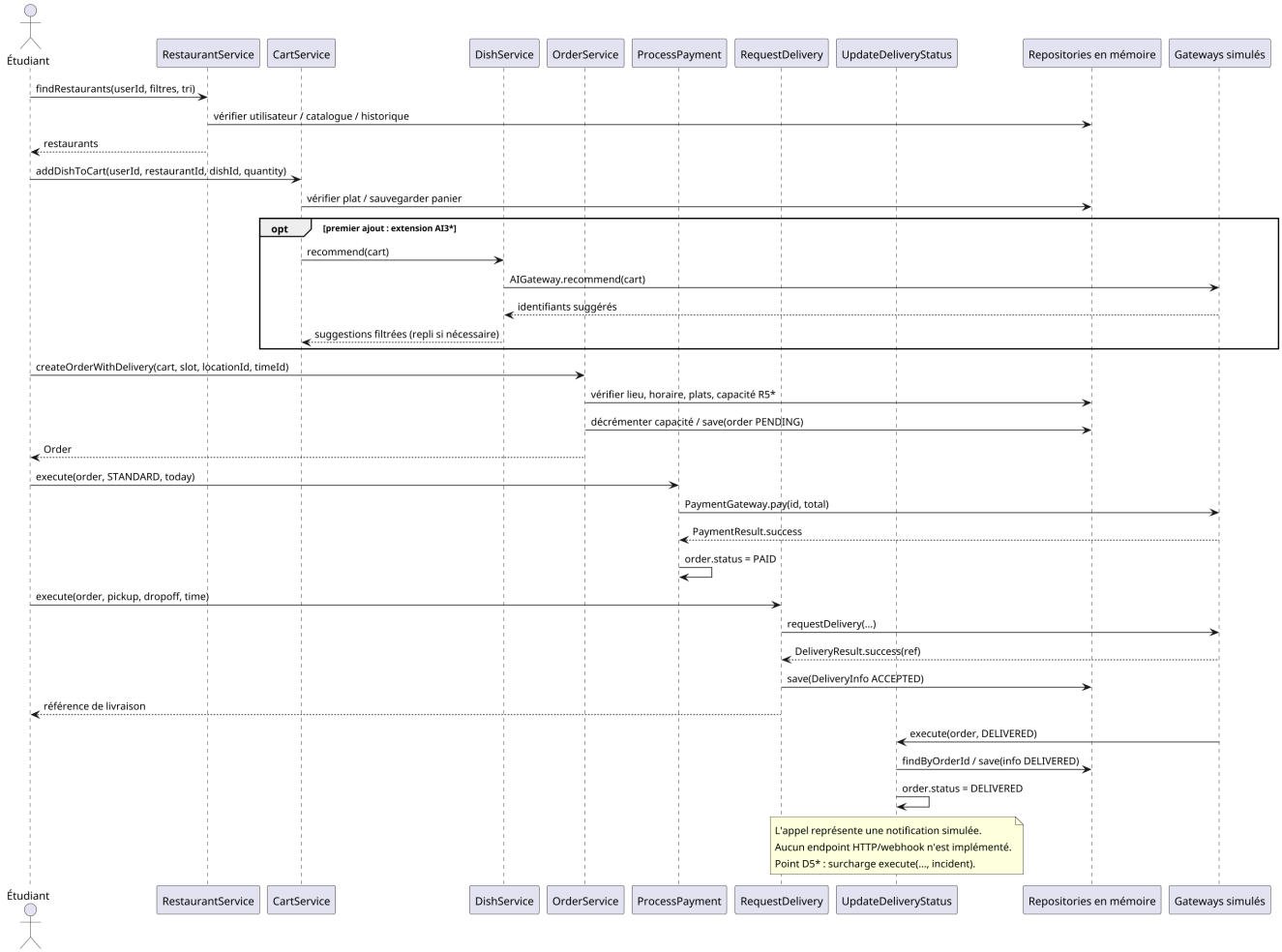
Les packages réels sont fr.unice.polytech.sophiaeats.{cart,delivery,dish,order,payment,restaurant,user}, chacun organisé en model, repository et/ou service. Le document ProjetArchitecture.md propose domain/usecases/infrastructure, mais cette structure n’est pas celle du code livré. Le rapport décrit l’organisation réellement implémentée.

Le cœur utilise des repositories injectés par constructeur. Les gateways PaymentGateway, ExternalDeliveryGateway et AIGateway portent les contrats des services simulés ; les exceptions de paiement, livraison et IA sont traitées à ces frontières. Les repositories en mémoire rendent les scénarios reproductibles sans infrastructure externe, avec perte des données à l’arrêt et absence de garantie transactionnelle.

RestaurantService, CartService, DishService et OrderService sont les points d’entrée applicatifs. Ils encapsulent des opérations métier ; aucune façade globale ne coordonne à elle seule commande, paiement et livraison. Ils constituent des entrées locales ; le code ne fournit pas la façade globale STEats prévue dans le D0.

Les classes nommées Adapter réalisent les contrats des gateways et permettent leur substitution. Elles ne traduisent pas une API incompatible réelle : aucun Adaptee externe n’est présent. Le patron GoF Adapter n’est donc pas formellement démontré ; il s’agit ici d’implémentations simulées de ports applicatifs. Le choix réduit la dépendance aux services tiers, mais ne prouve pas leur intégration réseau.

3.3 Diagramme de séquence unique — scénario nominal



Scénario nominal reconstitué : recherche, premier ajout avec recommandation, validation avec livraison, paiement, demande puis suivi de livraison. Ce parcours est rapproché des UC2, UC4 et UC5 du D0, adaptés à l’architecture actuelle ; le texte TD1 n’étant pas joint, sa correspondance exacte reste à confirmer. Il ne s’agit pas d’un test bout en bout existant : les tests fournis valident ces opérations par sous-parcours. Les appels des gateways et des notifications sont simulés.

4. Qualité du code, tests et gestion de projet

4.1 Focus sur le cœur fonctionnel

Indicateur demandé	Preuve disponible
Nombre de tests unitaires JUnit (SonarQube)	Non fourni. Surefire compte 93 tests JUnit, mêlant tests unitaires et intégration ; ce total ne prouve pas le nombre de tests purement unitaires.
Couverture du cœur (SonarQube)	Non fournie ; aucun taux de couverture vérifiable.
Capture SonarQube	Non fournie — preuve obligatoire à insérer ici.

Validation Maven locale : 126 tests réussis (93 JUnit + 33 scénarios Cucumber), 0 échec, 0 erreur, 0 ignoré ; demo-1.0-SNAPSHOT.jar produit. Source : journal Maven et rapports Surefire de l'archive develop analysée.

4.2 Traçabilité des tests d'intégration étroits

Composant / fonctionnalité	Nombre de tests d'intégration étroits	Nom d'un test clé utilisant Mockito	Comportement testé
Delivery Service	3 scénarios BDD de livraison	Absent : scénarios sans Mockito	Nominal : transmission, suivi DELIVERED ; refus d'une commande impayée.
Payment Service	9 JUnit ProcessPaymentTest	ProcessPaymentTest.gatewayExceptionIsIsolatedAndDoesNotBreakFlow	Défaillance : exception du gateway → résultat d'échec, statut PENDING conservé.
AI Recommandation	15 JUnit ciblés : 7 panier + 8 recommandation	Absent : doublures lambda, pas Mockito	Nominal et panne : filtrage, tentative unique, repli sans bloquer la commande.
Capacities & times	2 JUnit ciblés + 2 BDD ciblés	Absent : repositories en mémoire, pas Mockito	Nominal / saturation : configure le quota, refuse la commande au-delà.
Offers/Specials	0	Absent	Non implémenté.
Monitor delivery operations	0	Absent	Non implémenté.

Comptage : tests de frontière ou de collaboration locale, sans service distant réel. Les 4 tests ExternalAIAdapterTest sont exclus (adaptateur seul), ainsi que les 2 tests de panier sur fusion des quantités et protection des listes (hors recommandation). Les tests généraux de validation de commande sont exclus du comptage des capacités. Les tests Mockito exigés restent manquants pour livraison, IA et capacités.

4.3 Gestion de projet (2 lignes)

Archive fournie : branche develop ; workflows configurés pour main et/ou develop. Stratégie de branches, revues de PR et validation par un pair : à confirmer par l'équipe.

5. D5* — Incidents de livraison

5.1 Étudiant responsable

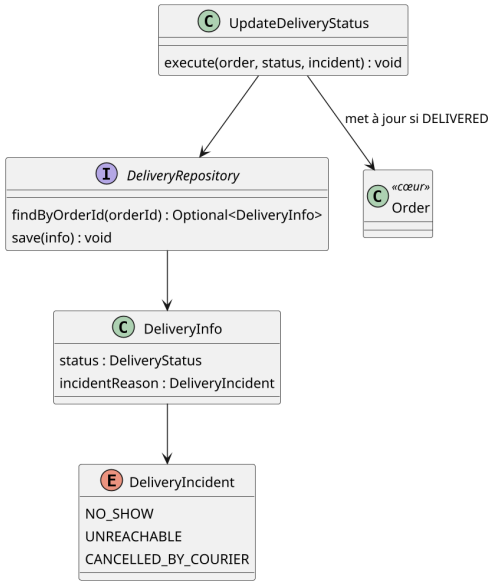
MOROU Essognina Fadilou — référent indiqué dans le D0 Team-J.

5.2 Stratégie métier

Conception observée : Le code représente une livraison échouée par DeliveryStatus.FAILED et un motif DeliveryIncident : NO_SHOW, UNREACHABLE ou CANCELLED_BY_COURIER. UpdateDeliveryStatus conserve l'état de commande PAID/READY ; il n'invente pas un nouvel OrderStatus pour un incident.

Spécifications — écarts par rapport à D0 : NO_SHOW devait entraîner clôture sans remboursement après 10 minutes ; faute du livreur et lieu inaccessible devaient entraîner annulation et remboursement intégral selon le moyen de paiement. Le D0 imposait aussi historique permanent horodaté et notification. Aucun de ces traitements n'est implémenté ; conserver PAID/READY constitue une réduction du périmètre, non une conformité.

5.3 Diagramme de classes



5.4 Connexion au cœur

UpdateDeliveryStatus.execute(Order, DeliveryStatus, DeliveryIncident) charge DeliveryInfo par orderId, met à jour le statut, enregistre le motif si FAILED puis sauvegarde la livraison. DELIVERED met aussi la commande à DELIVERED et efface le motif. OrderRepository n'est pas injecté : la modification de commande repose sur l'objet partagé en mémoire, sans sauvegarde explicite adaptée à un stockage distant.

5.5 Design Pattern

Aucun.

5.6 Qualité du code et des tests

Les step definitions contiennent des pas de panne et d'incident, mais les trois scénarios actifs DeliveryTraking.feature ne les utilisent pas. Aucun test direct de FAILED, du motif, de réponse nulle, de refus du fournisseur ou d'exception RequestDelivery n'a été identifié. Aucun contrôle des transitions, mécanisme de compensation, relivraison ou remboursement. D5* reste WIP.

6. R5* — Capacités et créneaux

6.1 Étudiant responsable

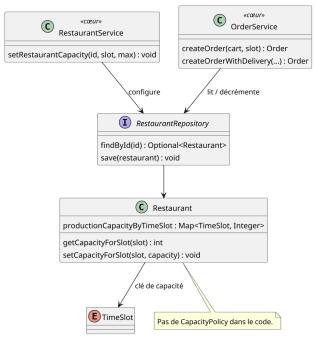
LOUBARIS Adam — référent indiqué dans le D0 Team-J.

6.2 Stratégie métier

Conception observée : Restaurant stocke un nombre de commandes restantes par TimeSlot dans une EnumMap. Tous les créneaux existent et ont une capacité initiale nulle. RestaurantService.setRestaurantCapacity exige un identifiant valide, un créneau et une capacité strictement positive.

Spécifications — écarts par rapport à D0 : deux stratégies étaient prévues (FixedSlotCapacityPolicy et PreparationTimeCapacityPolicy), via CapacityPolicy.computeEarliestReadyTime. Le code réalise uniquement un quota fixe restant par enum TimeSlot ; il ne calcule pas estimatedReadyTime, ne gère ni personnel ni temps de préparation, et ne transmet pas cette information à la livraison.

6.3 Diagramme de classes



Structure propre à R5* : Map<TimeSlot,Integer> dans Restaurant. Les deux politiques CapacityPolicy prévues en D0 sont absentes ; le diagramme représente le quota réellement implémenté.

6.4 Connexion au cœur

OrderService consulte la capacité avant de produire la commande, puis la décrémente de 1 et sauvegarde Restaurant. createOrder retourne null si le créneau est plein ; createOrderWithDelivery lance IllegalArgumentException. L’exigence est incorporée au parcours de commande : aucune interface de capacité substituable ne permet de la désactiver indépendamment.

6.5 Design Pattern

Aucun.

6.6 Qualité du code et des tests

RestaurantMenuAndCapacityTest.acceptsOrdersUntilCapacityIsFull valide saturation et refus ; CreateOrderWithDeliveryTest.rejectsWhenCapacityTimeSlotIsFull couvre le parcours avec livraison. Le BDD comprend la configuration de capacité et le scénario acceptation puis saturation. Les 4 tests de RestaurantMenuAndCapacityTest et les 8 de CreateOrderWithDeliveryTest couvrent aussi d’autres règles de menu et validation.

La lecture puis décrémentation n’est pas atomique ; les HashMap ne sont pas protégées contre les accès concurrents. La capacité est consommée avant paiement et n’est jamais restituée en cas d’échec. Si la sauvegarde de commande échoue, la capacité peut être perdue. Le setter du modèle peut accepter une valeur négative en contournant le service ; le service interdit de configurer zéro. Pas de calcul de temps de préparation ni de cohérence entre DeliveryTime et TimeSlot : état WIP.

7. R6* — Offres et promotions

7.1 Étudiant responsable

ATI-ATCHA Wahid — référent indiqué dans le D0 Team-J.

7.2 Stratégie métier

Conception observée : Aucun modèle d’offre, tarif promotionnel, règle de remise ou service de promotion n’est présent dans les classes de production. Le total est calculé dans OrderService par somme de quantity × unitPrice ; aucun passage par un moteur d’offres.

Spécifications — écarts par rapport à D0 : Campus Specials ciblés par plat/catégorie, type utilisateur, fenêtre de validité et quotas ; fidélité par couple utilisateur/restaurant, points après paiement, récompense et cumul dans un ordre précis. PromotionService, Promotion, CampusSpecial, LoyaltyProgram, Discount et LoyaltyAccount ne sont pas présents. Le module prévu est entièrement non réalisé.

7.3 Diagramme de classes

Néant : aucune structure propre à R6* implémentée dans l’archive.

7.4 Connexion au cœur

Aucune. OrderService calcule le total sans point d’extension pour les remises.

7.5 Design Pattern

Aucun.

7.6 Qualité du code et des tests

Aucun test JUnit, scénario Gherkin ou indicateur de qualité propre à R6* identifié. État BLOCKED. Une implémentation ultérieure devra définir les règles métier et leur cumul, puis valider le montant de commande et de paiement. Ce travail n’est pas présenté comme déjà réalisé.

8. A3* — Supervision des livraisons

8.1 Étudiant responsable

ADJEGAN Justice — référent indiqué dans le D0 Team-J.

8.2 Stratégie métier

Conception observée : Le suivi unitaire d'une livraison existe : DeliveryRepository.findByOrderId permet de consulter DeliveryInfo. Il ne constitue pas une supervision des opérations de livraison. Aucun service de monitoring administrateur, agrégation, tableau d'incidents ou liste globale n'a été identifié.

Spécifications — écarts par rapport à D0 : calcul des taux d'acceptation et d'échec, fenêtre de 20 livraisons avec minimum 5, seuil d'échec > 20 %, retard > 15 minutes, anti-spam et boîte administrateur. DeliveryMonitor, DeliveryStats, AlertRule et AdminInboxNotifier sont absents.

8.3 Diagramme de classes

Néant : aucun module propre à A3* implémenté.

8.4 Connexion au cœur

Aucune connexion de supervision. UpdateDeliveryStatus enregistre le suivi D4 et les informations D5*, mais n'émet pas d'événement vers un moniteur et n'expose pas de requête globale.

8.5 Design Pattern

Aucun.

8.6 Qualité du code et des tests

Aucun test spécifique de supervision ou d'agrégation. Les trois scénarios de suivi unitaire ne valident pas A3*. État BLOCKED ; la fonctionnalité devra définir les opérations observées, les acteurs autorisés et les données agrégées avant implémentation.

9. AI3* — Recommandations

9.1 Étudiant responsable

TSOULI Younes — référent indiqué dans le D0 Team-J.

9.2 Stratégie métier

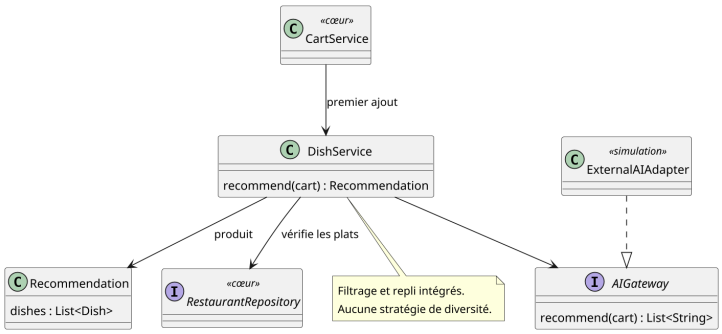
Conception observée : Cart.addItem retourne true uniquement au premier ajout dans un panier initialement vide et non encore traité. Le marqueur recommendationsAttempted est fixé avant l'appel IA ; vider le même panier ne réinitialise pas la tentative. CartService sauvegarde le panier avant de demander les suggestions à DishService.

DishService ne conserve que les plats du restaurant, actifs, en stock et absents du panier ; il déduplique les identifiants en conservant leur ordre. Si le gateway est absent, lève une exception ou ne fournit aucune suggestion valide, le repli renvoie tous les plats locaux éligibles dans l'ordre du menu. UserStories.md évoque catégorie/popularité : ce classement n'est pas implémenté.

Spécifications — écarts par rapport à D0 : le premier Dish d'une Order est devenu le premier ajout dans Cart ; recommandations limitées au même restaurant et hors panier ; réponse sans élément valide remplacée par le repli local. La diversité minimale évoquée par D0 n'est ni définie ni vérifiée, et les préférences utilisateur ne sont pas transmises. Le D0 présente une tension entre repli local et absence de suggestion lorsque toutes les réponses sont invalides ; le code retient le repli.

9.3 Diagramme de classes

Contrat : AIGateway.recommend(Cart) : List<String>. ExternalAIAdapter renvoie une liste prédéfinie et peut simuler un échec. Recommendation encapsule les Dish retenus. Le diagramme ci-dessous montre le branchement CartService → DishService → AIGateway et le modèle spécifique Recommendation.



9.4 Connexion au cœur

L'appel est optionnel et ne conditionne pas la création de commande. Le test failedAIIsNotRetriedAndDoesNotPreventOrderCreation exerce le repli, l'absence de deuxième tentative et la création d'une commande. Le code conserve également DishRecommendationService, dont la logique est proche de DishService ; les tests portant ce nom exécutent en réalité DishService.

9.5 Design Pattern

Aucun.

9.6 Qualité du code et des tests

9 tests CartServiceTest, 8 DishRecommendationServiceTest, 4 ExternalAIAdapterTest et 4 scénarios BDD. Les cas couvrent premier ajout, absence de relance, panne IA, références invalides et doublons, plats d'un autre restaurant, indisponibilité et absence d'alternative. Les doublures sont des lambdas/objets simulés, sans Mockito pour ce module. Limites : code dupliqué, pas de délai maximal d'appel, ni de classement métier du repli.

10. Auto-évaluation

10.1 Bilan qualitatif

Constats techniques vérifiables : l’organisation par domaines rend les services et modèles identifiables ; les gateways injectables permettent de tester les pannes sans dépendre d’un service tiers. Le panier et les recommandations disposent de tests ciblant la continuité de commande après panne IA.

Difficultés observables : la documentation d’architecture diffère du code, des documents et features restent incomplets, la validation des deux parcours de commande est asymétrique, et les cinq exigences stratégiques sont partielles ou absentes au regard du D0. Ces constats ne constituent pas un témoignage sur la collaboration ; l’équipe doit ajouter son bilan réel de l’organisation, des difficultés et des leçons tirées.

10.2 Répartition de l’implication (mécanisme des 100 points)

Membre	Points décidés par l’équipe	Coefficient
ADJEGAN Justice	100 pts	1,0
ATI-ATCHA Wahid	100 pts	1,0
LOUBARIS Adam	100 pts	1,0
MOROU Essognina Fadilou	100 pts	1,0
TSOULI Younes	100 pts	1,0
Total	500 / 500	—

Répartition de 100 points par membre explicitement communiquée par l’équipe : total 500 points, coefficient individuel 1,0. Cette décision collective n’est pas déduite de l’historique Git.